

Дополнительная лекция по Си № 8

12 мая 2025 г.

Коновалов А.В.

Продолжение про раздельную компиляцию

Мы остановились на том, что создали программу из трёх файлов `vec2d.c`, `main.c` и `vec2d.h`, для сборки которой используем следующий `make`-файл:

```
.PHONY: all clean

all: vec2d-example.exe

vec2d-example.exe: main.o vec2d.o
    gcc -o $@ $+

%.o: %.c vec2d.h
    gcc -c $<

clean:
    erase main.o vec2d.o vec2d-example.exe
```

Для сокращения дублирования кода мы уже познакомились с шаблонными правилами, которые использовали для сборки объектных файлов. Однако, есть способы уменьшить дублирование и далее. В частности, в сценарии сборки несколько раз упоминается имя исполнимого файла и список объектников. Их можно вынести в переменные — в этом случае при переименовании целевого файла или добавлении объектников изменения придётся вносить только один раз:

```
.PHONY: all clean

all: vec2d-example.exe

TARGET=vec2d-example.exe
OBJECTS=main.o vec2d.o

${TARGET}: ${OBJECTS}
    gcc -o $@ $+
```

```
%.o: %.c vec2d.h
gcc -c $<
```

```
clean:
    erase ${OBJECTS} ${TARGET}
```

Для определения переменной используется запись

<имя переменной>=<значение>

Для ссылки на переменную в тексте используется запись \${<имя переменной>}.

Как можем убедиться, цели all и clean работают:

```
D:\Си>mingw32-make
gcc -c main.c
gcc -c vec2d.c
gcc -o vec2d-example.exe main.o vec2d.o
```

```
D:\Си>mingw32-make clean
erase main.o vec2d.o vec2d-example.exe
```

В рассмотренном make-файле мы явно прописывали зависимость обоих объектных файлов от заголовочного файла vec2d.h. Но есть опция компилятора GCC -MM, которая заставляет компилятор создать список зависимостей для компилируемого объектного файла. Добавим её:

```
...
%.o: %.c vec2d.h
gcc -c $<
gcc -M $< > ${@}:.o=.d}
...
```

Компилятор GCC с опцией -M формирует список зависимостей и выводит его на стандартный вывод. Мы перенаправляем в файл с тем же именем, что и файл цели, но с расширением .d. Синтаксис \${<перем>:<подстрока>=<замена>} позволяет подставить значение переменной с автозаменой подстроки.

На самом деле есть встроенная переменная, которая означает базовую часть в шаблонном правиле (т.е. то, что сопоставляется с процентом), но лектор её забыл□

Содержимое файла зависимостей main.d следующее:

```
main.o: main.c \
C:/Program\ Files/CodeBlocks/MinGW/x86_64-w64-mingw32/include/stdio.h \
C:/Program\ Files/CodeBlocks/MinGW/x86_64-w64-mingw32/include/crtdefs.h \
C:/Program\ Files/CodeBlocks/MinGW/x86_64-w64-mingw32/include/_mingw.h \
C:/Program\ Files/CodeBlocks/MinGW/x86_64-w64-mingw32/include/_mingw_mac.h \
C:/Program\ Files/CodeBlocks/MinGW/x86_64-w64-mingw32/include/_mingw_secapi.h \
```

```

C:/Program\ Files/CodeBlocks/MinGW/x86_64-w64-mingw32/include/vdefs.h \
C:/Program\ Files/CodeBlocks/MinGW/x86_64-w64-mingw32/include/sdks/_mingw_directx.h \
C:/Program\ Files/CodeBlocks/MinGW/x86_64-w64-mingw32/include/sdks/_mingw_ddk.h \
C:/Program\ Files/CodeBlocks/MinGW/x86_64-w64-mingw32/include/_mingw_print_push.h \
C:/Program\ Files/CodeBlocks/MinGW/x86_64-w64-mingw32/include/_mingw_off_t.h \
C:/Program\ Files/CodeBlocks/MinGW/x86_64-w64-mingw32/include/swprintf.inl \
C:/Program\ Files/CodeBlocks/MinGW/x86_64-w64-mingw32/include/sec_api/stdio_s.h \
C:/Program\ Files/CodeBlocks/MinGW/x86_64-w64-mingw32/include/_mingw_print_pop.h \
vec2d.h

```

Фактически, здесь описывается цель с пререквизитами, которыми являются все заголовочные файлы, которые прямо или косвенно включаются в `main.c`.

Данные файлы зависимостей нужно включить в `make-файл`, чтобы утилита `Make` их учитывать:

```

.PHONY: all clean cleanall

all: vec2d-example.exe

TARGET=vec2d-example.exe
OBJECTS=main.o vec2d.o
DEPS=${OBJECTS:.o=.d}

-include ${DEPS}

${TARGET}: ${OBJECTS}
    gcc -o $@ $+

%.o: %.c
    gcc -c $<
    gcc -M $< > ${@:.o=.d}

clean:
    erase ${OBJECTS} ${TARGET}

cleanall: clean
    erase ${DEPS}

```

Во-первых, мы описали список файлов зависимостей как отдельную переменную:

```
DEPS=${OBJECTS:.o=.d}
```

Пользуемся уже знакомым синтаксисом замены в содержимом переменной.

Во-вторых, мы их включили в данный сценарий при помощи директивы `include`:

```
-include ${DEPS}
```

Знак «минус» в начале строки подавляет сообщение об ошибке, если файл отсутствует (такое может быть, если данные файлы ещё не созданы).

В-третьих, мы добавили ещё одну ложную цель `cleanall`, которая удаляет и файлы зависимостей (цель `clean` их оставляет).

```
.PHONY: all clean cleanall
```

...

```
cleanall: clean
    erase ${DEPS}
```

Заметим, что цель `cleanall` подразумевает также цель `clean`.

В-четвёртых, мы удалили явную зависимость от `vec2d.h` в шаблонном правиле для объектных файлов:

```
%.o: %.c
    gcc -c $<
    gcc -M $< > ${@:.o=.d}
```

Проверим цели `cleanall` и `all`:

```
D:\Си>mingw32-make cleanall
erase main.o vec2d.o vec2d-example.exe
erase main.d vec2d.d
```

```
D:\Си>mingw32-make
gcc -c main.c
gcc -M main.c > main.d
gcc -c vec2d.c
gcc -M vec2d.c > vec2d.d
gcc -o vec2d-example.exe main.o vec2d.o
```

Выполнение цели `cleanall` потребовало выполнения цели `clean`.

Изменим файл `vec2d.h` (добавим пустую строку) и посмотрим, что получится:

```
D:\Си>mingw32-make
gcc -c main.c
gcc -M main.c > main.d
gcc -c vec2d.c
gcc -M vec2d.c > vec2d.d
gcc -o vec2d-example.exe main.o vec2d.o
```

Выполнилась полная перекомпиляция, т.к. Make учла зависимости из `.d`-файлов.

Как видим, утилита Make — довольно низкоуровневый инструмент, её сценарий позволяет описывать зависимости между целями и команды для сборки целей, однако сами зависимости приходится описывать вручную. Поэтому на практике «голым» Make пользуются нечасто (но пользуются), создание `make`-файлов

автоматизируют при помощи других утилит: GNU autotools (autoconf, automake и т.д.), cmake (устанавливается вместе со фреймворком Qt), CMake и т.д. Часто среды разработки сами генерируют make-файл для проекта.

Инкапсуляция при использовании отдельной трансляции

В текущей версии библиотеки для работы с двумерными векторами представление вектора открыто: пользователи библиотеки (в нашем случае это `main.c`) знают, что вектор представлен в декартовой системе координат. Однако, реализацию можно скрыть ценой динамического выделения памяти.

Для этого в заголовочном файле структура должна быть только объявлена без определения, а операции с ней — принимать и возвращать экземпляры структуры по указателю. Перепишем соответствующим образом.

Переписанный файл `vec2d.h` имеет вид:

```
#ifndef VEC2D_H_
#define VEC2D_H_

struct Vec2d;

struct Vec2d *vec2d_create(double x, double y);
struct Vec2d *vec2d_add(const struct Vec2d *a, const struct Vec2d *b);
double vec2d_prod(const struct Vec2d *a, const struct Vec2d *b);
double vec2d_length(const struct Vec2d *v);
void vec2d_print(const struct Vec2d *v);
void vec2d_destroy(struct Vec2d *v);

#endif /* VEC2D_H_ */
```

В нём мы объявили структуру `Vec2d`, но её не определили — пользователи библиотеки не могут описать переменную типа `struct Vec2d`, могут только описывать указатели на эту структуру, объявлять функции, принимающие и возвращающие эту структуру и всё. Без определения структуры нельзя определить её объём, а значит, создать переменную соответствующего типа.

Последующие функции работают только с указателями на структуру.

Функция, принимающая указатель на структуру, потенциально эту структуру может изменить. Поэтому для функций, которые по смыслу принимают структуру «по значению» мы указываем тип аргумента как `const struct Vec2d *`, подчёркивая, что они не меняют свой аргумент. Ранее эти функции принимали структуру по значению, а значит, получали копию структуры.

Также добавили две функции: `vec2d_create` и `vec2d_destroy`, первая выделяет память и инициализирует структуру, вторая уничтожает принятую структуру.

Использование `vec2d_destroy` инкапсулирует тот факт, что для выделения памяти используется `malloc` — может быть использован и альтернативный аллокатор.

Определение структуры ушло в `vec2d.c`, определения операций в нём очевидны:

```
#include <math.h>
#include <stdio.h>
#include <stdlib.h>
#include "vec2d.h"

struct Vec2d {
    double x;
    double y;
};

struct Vec2d *vec2d_create(double x, double y)
{
    struct Vec2d *p = malloc(sizeof(*p));
    *p = (struct Vec2d){ .x = x, .y = y };
    return p;
}

struct Vec2d *vec2d_add(const struct Vec2d *a, const struct Vec2d *b)
{
    return vec2d_create(a->x + b->x, a->y + b->y);
}

double vec2d_prod(const struct Vec2d *a, const struct Vec2d *b)
{
    return a->x * b->x + a->y * b->y;
}

double vec2d_length(const struct Vec2d *v)
{
    return sqrt(vec2d_prod(v, v));
}

void vec2d_print(const struct Vec2d *v)
{
    printf("%.1f,%.1f", v->x, v->y);
}

void vec2d_destroy(struct Vec2d *v)
{
    free(v);
}
```

Пользователь библиотеки, функция `main()` для создания и уничтожения векторов использует соответствующие функции, остальное не изменилось. Файл `main.c`:

```
#include <stdio.h>
#include "vec2d.h"

int main()
{
    struct Vec2d *a = vec2d_create(1, 2);
    struct Vec2d *b = vec2d_create(2, 2);
    struct Vec2d *ab = vec2d_add(a, b);
    double ab_len = vec2d_length(ab);
    printf("a = ");
    vec2d_print(a);
    printf("\nb = ");
    vec2d_print(b);
    printf("\nab = ");
    vec2d_print(ab);
    printf(", length = %.1f", ab_len);
    vec2d_destroy(a);
    vec2d_destroy(b);
    vec2d_destroy(ab);
    return 0;
}
```

Заметим, что при изменении реализации структуры (например, использование не декартовой, а полярной системы координат) файл `main.c` не изменится и его даже не потребуется перекомпилировать.

Небольшая демонстрация автоматического построения зависимостей

Вынесем из файла `vec2d.c` определение структуры в отдельный заголовочный файл (смысла это не несёт, просто для демонстрации):

Файл `vec2d_def.h`:

```
#ifndef VEC2D_DEF_H_
#define VEC2D_DEF_H_

struct Vec2d {
    double x;
    double y;
};

#endif /* VEC2D_DEF_H_ */
```

В файл `vec2d.c` вставлен соответствующий `#include`:

```
...
#include "vec2d_def.h"
...
```

Перекомпилируем проект:

```
D:\Си>mingw32-make
gcc -c main.c
gcc -M main.c > main.d
gcc -c vec2d.c
gcc -M vec2d.c > vec2d.d
gcc -o vec2d-example.exe main.o vec2d.o
```

Теперь, если файл `vec2d_def.h` изменится, то перекомпилируется только `vec2d.o` (и соответствующий файл зависимостей `vec2d.d`):

```
D:\Си>mingw32-make
gcc -c vec2d.c
gcc -M vec2d.c > vec2d.d
gcc -o vec2d-example.exe main.o vec2d.o
```

В файле зависимостей `vec2d.d` ссылки на оба заголовочных файла:

```
vec2d.o: vec2d.c \
C:/Program\ Files/CodeBlocks/MinGW/x86_64-w64-mingw32/include/math.h \
...
C:/Program\ Files/CodeBlocks/MinGW/x86_64-w64-mingw32/include/stdio.h \
...
C:/Program\ Files/CodeBlocks/MinGW/x86_64-w64-mingw32/include/stdlib.h \
...
vec2d.h vec2d_def.h
```

В листинге выше куча служебных заголовочных файлов (косвенно подключаемых из `math.h`, `stdio.h` и `stdlib.h`) опущена для краткости. Нас интересует последняя строчка с двумя нашими заголовочными файлами.

Таким образом, если файлы зависимостей создаются автоматически, то нам не требуется изменять `make`-файл при добавлении новых заголовочных файлов и явного указания того, какие объектные от каких заголовочных зависят.

Использование `make` для создания документации

Утилита `Pandoc` позволяет конвертировать файлы в разметке `Markdown` в `PDF`, программа `Graphviz` может из файлов описания графа создавать файлы графических форматов (`.png`, `.svg` и т.д.).

Поэтому для создания документации с картинками можно использовать примерно такой сценарий:


```

.PHONY: all clean

all: book.pdf

CHAPTERS=chapter1.md chapter2.md chapter3.md
PICS=fig1.png fig2.png fig3.png fig4.png
PANDOC_FLAGS=--pdf-engine=xelatex \
    --mainfont="Comic Sans MS" \
    --monofont="Lucida Console"

book.pdf: ${CHAPTERS} ${PICS}
    pandoc ${PANDOC_FLAGS} -o $@ ${CHAPTERS}

%.png: %.dot
    dot $< -Tpng -o$@

clean:
    rm -f book.pdf ${PICS}

```